

Anwendungsorientierte Softwareentwicklung

Laborblatt 10: Jetzt mit bewegten Bildern!

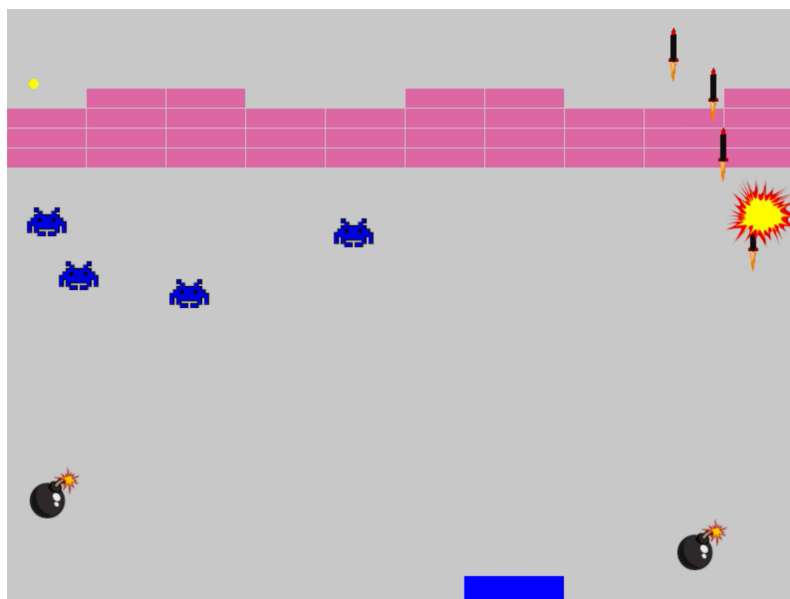
Lernziele

- **Refactoring (Aufräumen gehört immer dazu)**
- **Exceptions abfangen**
- **Vermeidung von Datenstaus beim Auslesen der seriellen Schnittstelle**
- **Und vor allem: Bitmaps und neue Spielfeatures**

Ziel für heute

- Der Controller sollte angeschlossen und benutzt werden können, auch wenn das Spiel bereits läuft. Man sollte ihn aber auch im laufenden Betrieb entfernen können. Das wird bisher schon teilweise unterstützt, aber eben nur teilweise.
- Wenn man am Potentiometer schnell dreht und gleichzeitig den Taster verwendet, kann es passieren, dass das Spiel „laggt“ (ja, man sagt das so!). Die Daten, die man vom Controller bekommt, stauen sich. Wenn man plötzlich nicht mehr am Potentiometer dreht, bewegt sich das Paddle trotzdem noch eine Weile.
- Zu einem Spiel gehört natürlich auch Sound, z. B. wenn der Ball abprallt, ein Raumschiff abgeschossen wird usw.
- Eine Rakete sollte wie eine Rakete aussehen, ein Raumschiff wie ein Raumschiff. Kurz gesagt: Manche Dinge will man mit Bitmaps darstellen und nicht nur mit geometrischen Objekten wie Kreisen oder Rechtecken.
- Und dann braucht es natürlich viele neue Features, zum Beispiel dass Raumschiffe Bomben abwerfen können, Bricks mehrfach getroffen werden müssen usw. Und vielleicht sollte man auch nicht unendlich viel Munition haben.
- In diesem Blatt wird an ein paar Beispielen gezeigt, wie solche neuen Features realisiert werden können – damit ihr anschließend auch selbstständig eigene Ideen umsetzen könnt.

Wir werden diese Dinge in einer scheinbar willkürlichen Reihenfolge angehen. Das soll euch zeigen, wie so ein typischer Ablauf in der Praxis aussieht: Man nimmt sich zunächst nur vor, eine Verbesserung einzufügen – und dann kommt man von einem zum anderen. Man entdeckt Stellen im Code, die man umorganisieren sollte, damit sich neue Features besser einfügen lassen. Man findet Bugs, die bisher verborgen waren. Und man bekommt Ideen für das nächste Feature, das man eigentlich gar nicht auf dem Schirm hatte.



Vorbereitung: Teil 1

Wie üblich arbeiten wir mit zwei Terminals: **links**: Quellcode editieren, **rechts**: Code kompilieren, Spiel testen. Repository klonen mit

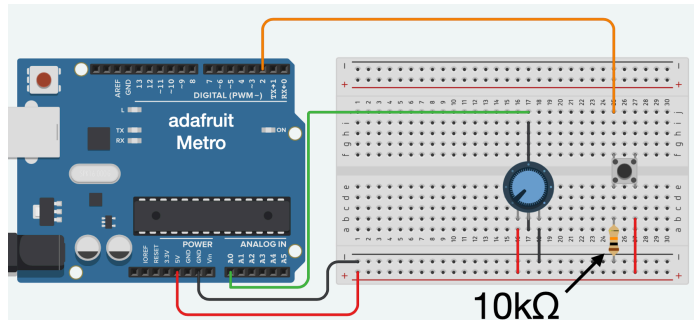
```
git clone https://gitlab.uni-ulm.de/BENUTZERNAME/asp-ulm
```

Dann in beiden Terminals ins Verzeichnis mit dem Python-Code wechseln:

```
cd asp-ulm/python
```

Vorbereitung: Teil 2

Ihr solltet das Breadboard mit dem bekannten Schaltkreis aus Taster und Potentiometer aufbauen und korrekt an den Mikrocontroller anschließen. Der Taster ist am digitalen Eingang Pin 2 angeschlossen, das Potentiometer hängt am analogen Eingang A0. Rechts seht ihr ein Bild mit dem vollständigen Aufbau.



Vorbereitung: Teil 3

Im ersten Schritt sollen heute die Raketen nicht mehr als einfache Kreisscheiben dargestellt werden, sondern durch ein Bild. Damit ihr die Bilder nicht selbst zeichnen müsst, habe ich dafür ein Git-Repository mit Zusatzmaterial erstellt. **Jetzt ist es allerdings etwas tricky**, wie ihr es schafft, dass ihr diese Dateien am Ende eurem eigenen Repository hinzufügen könnt. Dazu gibt es diese Anleitung:

1. Stellt sicher, dass ihr euch im Unterverzeichnis eures Projekts befindet. Gebt im Terminal `pwd` ein. Damit wird das aktuelle Verzeichnis ausgegeben.
An den Laborrechnern sollte das `/home/asp/asp-ulm/python` sein.

2. Klont mein Repository mit dem Zusatzmaterial:

```
git clone https://github.com/michael-lehn/asp-ulm-data.git
```

Damit gibt es jetzt ein neues Unterverzeichnis `asp-ulm-data`.

Bestätigt das, indem ihr mit `ls` den Inhalt des aktuellen Verzeichnisses anzeigen lasst.

3. Benennt den Ordner in `data` um: `mv asp-ulm-data data`

Lasst euch wieder mit `ls` anzeigen, was im aktuellen Verzeichnis liegt, und bestätigt, dass der Unterordner jetzt `data` heißt.

4. Jetzt kommt der heikle Teil, weil etwas gelöscht wird. Lasst euch erst mit

```
ls -a data
```

anzeigen, was im Ordner alles drin ist. Es sollten nützliche Dinge drin sein, wie Bild- und Sounddateien, aber auch ein Verzeichnis `.git`. Und genau dieses `.git` muss weg! Das enthält Verwaltungsdateien, weil dieses Verzeichnis ursprünglich aus meinem Repository geklont wurde. Dieses Zeug wollt ihr nicht mitschleppen.

Führt dazu aus:

```
rm -rf data/.git
```

Wenn ihr danach wieder

```
ls -a data
```

ausführt, sollte `.git` verschwunden sein.

5. Jetzt fügt ihr den Ordner mit dem gesamten Inhalt in euer eigenes Repository hinzu:

```
git add data/*
git commit -a -m "Bilder und Sound-Files"
git push
```

Schritt 1: Jetzt geht's endlich los – mit Bildern!

Schliesst den Mikrocontroller an und führt `usb_check` aus.

Öffnet dann die Datei `ship.py` mit der Klasse `Ship`, die für die Raumschiffe zuständig ist.

Diese sollen jetzt nicht mehr als einfache Kreisscheiben dargestellt werden, sondern so aussehen wie bei Space Invaders.

Im Konstruktor (also der `__init__`-Methode) fügt ihr folgende neue Zeile ein:

```
self.img = pygame.image.load("data/space_invader0.png")
```

Damit hat jedes Ship-Objekt ein neues Attribut `img`, das mit einem Bild-Objekt initialisiert wird. Und das Bild-Objekt wurde erzeugt, indem die Datei `data/space_invader0.png` geladen wurde.

Jetzt sollte dieses Bild gezeichnet werden statt des Kreises. Also ändert die Methode `draw` so ab, dass sie nur noch diese Zeile enthält:

```
pygame.Surface.blit(Game.screen, self.img, (self.x, self.y))
```

Die `blit`-Methode kopiert ein Bild (genauer: ein `Surface`-Objekt) an eine bestimmte Position auf eine andere Surface, hier also auf den Bildschirm. Damit könnt ihr beliebige Bilder statt einfacher Formen darstellen.



Fun Fact: Woher kommt „BitBlit“?

Der Begriff **BitBlit** (bit block transfer) stammt aus den legendären Xerox PARC Forschungslabors der 1970er Jahre. Dort wurde die grafische Benutzeroberfläche (GUI) entwickelt, wie wir sie heute kennen – mit Fenstern, Icons, Mauszeiger. Damit das alles flüssig dargestellt werden konnte, brauchte man eine schnelle Methode, um Bilddaten von einer Speicherstelle an eine andere zu kopieren. Das war die Geburtsstunde der **BitBlit-Funktion** – und das Wort **blit** hat sich bis heute als Kurzform in vielen Grafikbibliotheken (wie Pygame) gehalten.

Aufgabe

1. Testet das neue Feature indem ihr `python breakout.py` ausführt.
2. Wie oben erwähnt, ist `blit()` eine Methode – und zwar der Klasse `pygame.Surface`. Sowohl `self.img` als auch `Game.screen` sind Objekte dieser Klasse. Ändert jetzt die Zeile in der `draw`-Methode ab zu:

```
Game.screen.blit(self.img, (self.x, self.y))
```

Damit passiert **genau dasselbe** wie zuvor. Intern wird es von Python quasi in das umgeschrieben, was vorher dastand. In der neuen Form ist aber klarer, was gemeint ist: **Auf das Game.screen-Objekt wird die blit-Methode angewandt, um das self.img-Objekt an die Position (self.x, self.y) zu kopieren.**

3. Wir haben noch einen kleinen Schönheitsfehler: Die Bilder der Space Invader werden aktuell so gezeichnet, dass `(self.x, self.y)` die linke obere Ecke des Bildes ist. Konkret bedeutet das z. B., dass Raketen scheinbar „durchfliegen“, wenn man das Bild rechts mit einer Rakete trifft. Gleichzeitig werden die Space Invader schon abgeschossen, wenn die Rakete eigentlich links daran vorbeifliegt.

Testet das! Wir wollen keine Probleme lösen, die wir nicht genau kennen.

4. Um das zu korrigieren, ändert den Code in der `draw`-Methode ab zu:

```
pos = self.img.get_rect()
pos.center = (self.x, self.y)
Game.screen.blit(self.img, pos)
```

Die `get_rect()`-Methode liefert ein Objekt vom Typ `pygame.Rect`.

Solche Objekte stellen Rechtecke dar (in diesem Fall die Umrandung des Bildes) und können zentriert ausgerichtet werden, indem man deren `center`-Attribut überschreibt.

Testet, ob damit das Problem gelöst ist. Wenn ja, dann:

```
git commit -a -m "Bild für Space Invader"
git push
```

Schritt 2: Mehr Bilder

Öffnet die Datei `rocket.py` mit der Klasse `Rocket`, die für die Raumschiffe zuständig ist. Die sollen jetzt natürlich aussehen wie richtige Raketen!

Aufgabe

1. Ihr habt gesehen, wie es bei den Raumschiffen ging. Macht es jetzt analog bei den Raketen und verwendet dafür das Bild: `data/rocket0.png`.
2. Wenn ihr das Konzept direkt übertragen habt, dann sind die Bilder der Raketen aktuell an deren Zentrum ausgerichtet.

Bei Raketen ist das eigentlich nicht ganz richtig – wie beim LötKolben ist nämlich die **Spitze** das gefährliche Ding. Hier ist es allerdings schwieriger zu testen, ob ein Space Invader nur dann getroffen wird, wenn er (aktuell) wirklich von der Raketenmitte erwischt wird. Lasst uns daher die Spielgeschwindigkeit kurzfristig etwas drosseln, damit man solche Details besser sehen kann.

Ändert in `breakout.py` die Zeile:

```
time.sleep(1/30)
```

um in:

```
time.sleep(1)
```

Dann wird pro Sekunde nur ein Frame gezeichnet – wir sehen also alles in Zeitlupe. Wenn ihr jetzt das Spiel startet, solltet ihr sofort sehen, dass beim Start einer Rakete diese bereits zur Hälfte aus dem Paddle (unserer „Abschussrampe“) herausragt.

Ändert nun in `rocket.py` die Zeile:

```
pos.center = (self.x, self.y)
```

um in:

```
pos.midtop = (self.x, self.y)
```

Wenn ihr jetzt das Spiel neu startet, seht ihr, dass die Rakete an ihrer Spitze (dem gefährlichen Ding) ausgerichtet ist.

3. Da ihr das Spiel gerade in Zeitlupe ausführt, fällt euch vielleicht ein Problem auf, das zuvor verborgen war: Die Steuerung ist träge – man sagt, sie **lagt** (von engl. *lag* = Verzögerung). Wenn man schnell am Potentiometer hin und her dreht und dann loslässt, bewegt sich das Paddle trotzdem noch eine Weile weiter hin und her.

Das ist ein typisches Problem, wenn Daten mit unterschiedlicher Frequenz produziert und verarbeitet werden (in der Elektronik nennt man das *crossing clock domains*).

Darum kümmern wir uns im nächsten Schritt.

Wenn ansonsten alles tut (und ihr das Problem, das wir im nächsten Schritt lösen, beobachtet habt), dann:

```
git commit -a -m "Raketen bild"
git push
```

Schritt 3: Daten vom Controller schneller verarbeiten

Dort ist der Code, mit dem Daten von der seriellen Schnittstelle (über unser Modul `controller`) gelesen und verarbeitet werden.

Der sollte in etwa so aussehen:

```
ser_data = controller.readdata()
if ser_data:
    if ser_data[0] == "A":
        rockets.append(Rocket(Paddle.x + Paddle.w // 2, Paddle.y, -10))
    elif ser_data[0] == "X":
        Paddle.set(ser_data[1] * 1.5)
```

Das Problem: Dieser Code wird **nur einmal pro Frame** ausgeführt, die serielle Schnittstelle liefert aber Daten **viel schneller**.

Das führt dazu, dass sich Daten „stauen“, wie ihr im letzten Schritt gesehen habt.

Bei uns lässt sich das Problem zum Glück sehr leicht lösen, indem wir den Code abändern in:

```
while True:
    ser_data = controller.readdata()
    if ser_data is None:
        break
    if ser_data[0] == "A":
        rockets.append(Rocket(Paddle.x + Paddle.w // 2, Paddle.y, -10))
    elif ser_data[0] == "X":
        Paddle.set(ser_data[1] * 1.5)
```

Damit werden die Daten quasi sofort verarbeitet: Die Schleife liest so lange Daten, bis keine neuen mehr da sind, und verarbeitet sie direkt – z. B. indem neue Raketen erzeugt oder die Paddle-Position gesetzt wird.

Aufgabe:

1. Testet, dass durch diese Änderung das Spiel zwar **immer noch sehr träge** ist – aber nur, weil wir nach wie vor **nur einen Frame pro Sekunde** zeichnen.
Die Verzögerungen aufgrund der Datenverarbeitung sind aber weg.
Wenn man jetzt schnell am Potentiometer dreht und dann aufhört, wird im nächsten Frame das Paddle **sofort an der neuen Position** gezeichnet und bleibt dann dort stehen.
2. Natürlich wollen wir jetzt wieder mit mindestens 30 Frames pro Sekunde spielen.
Aber wir machen das diesmal richtig und verwenden nicht mehr `time.sleep()`, sondern die dafür von Pygame vorgesehene Clock.
Man verwendet sie nach diesem Muster:

```
clock = pygame.time.Clock()
while running:
    # hier kommt der Code, um Ereignisse abzufragen
    # und einen Frame zu zeichnen

    # und am Ende der Schleife:
    clock.tick(30)
```

Wendet dieses Muster an und testet, dass das Spiel jetzt wieder flüssig läuft – eventuell sogar flüssiger als zuvor.

Was steckt dahinter?

- Vor der Hauptschleife wird ein Clock-Objekt angelegt. Dieses Objekt speichert intern die aktuelle Zeit.
- Wenn man die Methode tick(30) aufruft, prüft sie:
 - **Wenn seit der gespeicherten Zeit noch keine 1/30 Sekunde vergangen ist**, blockiert sie kurz, bis die Zeitspanne vorbei ist, und speichert dann die neue aktuelle Zeit.
 - **Wenn schon genug Zeit vergangen ist**, speichert sie einfach nur die aktuelle Zeit und macht sofort weiter.

Was ist der Vorteil?

Das Zeichnen dauert nicht immer exakt gleich lang!

Wie lange es dauert, hängt z. B. davon ab, wie viele Space Invader oder Bälle gerade im Spiel sind, oder ob gerade ein Objekt getroffen wurde.

In der Regel dauert das Zeichnen aber deutlich **weniger** als 1/30 Sekunde.

Durch diesen Mechanismus wird sichergestellt, dass ein Frame **immer nach der gleichen Zeitspanne** neu gezeichnet wird – unabhängig davon, wie viel gerade los ist.

3. Wenn alles tut, ab ins Repository damit:

```
git commit -a -m "Lag beseitigt und Clock verwendet"
git push
```

Schritt 4: Explosionen

Nach der langweiligen Datenverarbeitung brauchen wir Action – die Space Invader sollen explodieren, wenn sie getroffen werden!

Zufälligerweise gibt es mit `data/explosion.png` dafür schon ein passendes Bild.

Der erste Ansatz soll sein, dass in der `draw()`-Methode der Ship-Klasse abgefragt wird, ob das Attribut `alive` den Wert `True` oder `False` hat.

Ist es `True`, wird das „normale“ Bild angezeigt – ansonsten das mit der Explosion.

Öffnet die Datei `ship.py` und fügt im Konstruktor (also der `__init__`-Methode) die Zeile hinzu:

```
self.die = pygame.image.load("data/explosion.png")
```

Ändert dann die `draw`-Methode so ab:

```
def draw(self):
    if self.alive:
        img = self.img
    else:
        img = self.die
    pos = img.get_rect()
    pos.center = (self.x, self.y)
    Game.screen.blit(img, pos)
```

Aufgabe

1. Macht die Änderung und testet, ob das funktioniert. Ihr solltet feststellen, dass das Programm zwar **ohne Fehler** durchläuft, aber **keine Explosionen angezeigt werden**.

2. **Warum werden keine Explosionen angezeigt?**

Das Problem ist: Ein Raumschiff wird gar nicht mehr gezeichnet, wenn sein `alive`-Attribut `False` ist.

Schaut euch die Datei `breakout.py` an – dort taucht in der Hauptschleife eine Zeile wie diese auf:

```
ships = [item for item in ships if item.alive]
```

Hier ist `ships` die Liste der Raumschiffe, die gezeichnet werden. Wenn nun ein Schiff getroffen wurde, wird es **sofort** aus der Liste entfernt – und erscheint deshalb auch nicht mehr für einen letzten „Explosionen-Frame“.

Das Problem lösen wir so:

(a) Treffer anders markieren

Findet in `breakout.py` die Stelle, an der bisher bei einem Treffer steht:

```
s.alive = False
```

Ändert das ab zu:

```
s.hit = True
```

Tipp: Testet gleich, ob das Programm noch ohne Fehler durchläuft – so erkennt ihr Tippfehler frühzeitig. Wundert euch aber nicht: Jetzt sind die Space Invader erstmal unverwundbar – wir sind noch nicht fertig!

(b) hit-Attribut initialisieren und nutzen

In `ship.py` initialisiert ihr im Konstruktor das neue Attribut:

```
self.hit = False
```

In der `draw`-Methode ändert ihr den Code so ab:

```
def draw(self):
    if self.hit:
        img = self.die
        self.alive = False
    else:
        img = self.img
        pos = img.get_rect()
        pos.center = (self.x, self.y)
        Game.screen.blit(img, pos)
```

Testet, ob jetzt Explosionen angezeigt werden!

Ein getroffener Space Invader sollte **für genau einen Frame** noch als Explosion sichtbar bleiben, bevor er aus der Liste (und dem Leben) verschwindet.

3. Nachdem euch jetzt eine Lösung vorgekaut wurde, dürft ihr mal wieder was selber lösen! 😊

Wenn eine Rakete einen Space Invader trifft, soll **auch die Rakete explodieren**.

Wenn ein Ball den Space Invader trifft, soll der Ball **nicht** explodieren.

Ein möglicher Lösungsweg (analog zur vorigen Teilaufgabe):

- (a) In `breakout.py` wird nicht nur beim getroffenen Schiff das Attribut `hit` gesetzt, **sondern auch bei dem Objekt, das den Treffer verursacht hat**.

- (b) In der `Ball`-Klasse wird das `hit`-Attribut ignoriert.

In der `Rocket`-Klasse wird es **so beachtet wie in der Ship-Klasse**.

4. Wenn alles funktioniert, dann ab ins Repository damit:

```
git commit -a -m "Explosionen"
git push
```

Schritt 5: Code aufräumen

Schaut euch die Datei `wall.py` an. Da ist es aktuell ziemlich unschön, dass Dinge wie die Breite und Höhe der Bricks und auch die maximale Anzahl an Bricks pro Zeile und Spalte **hart codiert** als Konstanten vorkommen.

Wahrscheinlich geht es euch wie mir: Ich musste das eine Weile anschauen, um zu erkennen, was genau festgelegt wurde. Und das habe ich für euch entschlüsselt:

- Ein Brick ist **80 Pixel breit** und **20 Pixel hoch**.
- Es gibt maximal **10 Brick-Spalten** und **30 Brick-Zeilen**.

Schaut euch außerdem im anderen Terminal die Datei `game.py` an.

Da wird in der `init`-Methode ebenfalls hart codiert die Größe der Zeichenfläche festgelegt:

800 Pixel breit, 600 Pixel hoch.

Zum Glück passt das zusammen, so wie es aktuell festgelegt ist:

- 80 Pixel × 10 Spalten → 800 Pixel breit.
- 20 Pixel × 30 Zeilen → 600 Pixel hoch.

Prima, dann ist doch alles gut?

Ja, schon. Aber was, wenn man die Größe der Bricks ändern will? Oder mehr Bricks haben will? Oder was, wenn man – wie ich – den Code auch noch in ein paar Wochen verstehen will?

Lasst es uns besser organisieren! Ändert `game.py` so ab:

```
import pygame

class Game:
    width, height = None, None
    screen = None
    brick_rows, brick_cols = 30, 10
    brick_width, brick_height = 80, 20

    def init():
        Game.width = Game.brick_cols * Game.brick_width
        Game.height = Game.brick_rows * Game.brick_height
        pygame.init()
        Game.screen = pygame.display.set_mode((Game.width, Game.height))

    def quit():
        pygame.quit()
```

Jetzt wird an einer zentralen Stelle festgelegt, wie groß die Bricks sind und wie viele Zeilen und Spalten es maximal gibt.

Aufgabe

1. Jetzt seid ihr gefragt, in `wall.py` von den neuen `Game`-Attributen Gebrauch zu machen:
 - Ersetzt jedes Vorkommen von `30` mit `Game.brick_rows`.
 - Ersetzt jedes Vorkommen von `10` mit `Game.brick_cols`.
 - Ersetzt jedes Vorkommen von `80` mit `Game.brick_width`.
 - Ersetzt jedes Vorkommen von `20` mit `Game.brick_height`.
 Testet danach, ob noch alles funktioniert!
2. Prüft außerdem, ob man die neuen Konstanten in `game.py` ändern kann und trotzdem noch alles funktioniert. Ändert zum Beispiel `Game.brick_cols` auf `15`. Testet, ob das Spiel dann noch tut.
3. Macht danach die Änderung wieder rückgängig – eine Zeichenfläche von 800 × 600 Pixel ist nicht schlecht, wenn man das Spiel später mal auf einem Raspberry Pi spielen will. 😊
4. Wenn alles klappt:


```
git commit -a -m "brick konstanten"
git push
```

Schritt 6: Noch mehr Code aufräumen

Schaut euch die Datei `breakout.py` an.

Die Hauptschleife wird langsam ziemlich umfangreich, denn aktuell passiert darin alles auf einmal:

- Wir fragen die serielle Schnittstelle nach neuen Daten ab.
- Wir fragen Pygame nach neuen Key-Events ab.
- Wir zeichnen und bewegen Bälle, Raumschiffe und Raketen.
- Wir prüfen, ob es Kollisionen gibt.

Jetzt soll zumindest das **Zeichnen und Bewegen** von Bällen, Raumschiffen und Raketen besser organisiert werden.

Warum?

- (a) Der Code ist fast identisch.
- (b) Wenn die Raumschiffe später auch noch Bomben abwerfen, kommt eine weitere Liste von Objekten dazu, die man bewegen und zeichnen muss.

Also: **Jetzt ist der richtige Zeitpunkt, das sauber zu machen!**

Und wir machen es gleich ganz richtig:

Die Listen der Objekte, die bewegt und gezeichnet werden – also `balls`, `rockets` und `ships` – verschieben wir in die Game-Klasse.

Dort wird es außerdem eine Methode `draw()` geben, die alle diese Listen verwaltet.

Ändert `game.py` so ab:

```
import pygame

class Game:
    width, height = None, None
    screen = None
    brick_rows = 30
    brick_cols = 15
    brick_width = 80
    brick_height = 20
    balls = []
    rockets = []
    ships = []

    def init():
        Game.width = Game.brick_cols * Game.brick_width
        Game.height = Game.brick_rows * Game.brick_height
        pygame.init()
        Game.screen = pygame.display.set_mode((Game.width, Game.height))

    def draw():
        for item in Game.balls + Game.rockets + Game.ships:
            item.draw()
            item.move()
        Game.balls = [item for item in Game.balls if item.alive]
        Game.rockets = [item for item in Game.rockets if item.alive]
        Game.ships = [item for item in Game.ships if item.alive]

    def quit():
        pygame.quit()
```

Aufgaben

1. Passt `breakout.py` so an, dass die Änderung wirksam ist:
 - Entfernt den Code, der bisher die Bälle, Raumschiffe und Raketen bewegt und zeichnet. Stattdessen soll jetzt einmalig `Game.draw()` aufgerufen werden.
 - Alle Stellen, wo noch direkt auf die Listen `balls`, `rockets` oder `ships` zugegriffen wird, müssen jetzt ebenfalls angepasst werden. Schreibt dort z. B. statt `balls` → `Game.balls` usw.
 - Testet danach, ob wieder alles funktioniert!
2. Damit ihr seht, dass dieses Aufräumen auch nützlich ist, macht noch eine kleine Änderung in `wall.py`: Immer wenn ein Brick von einem Ball getroffen wird, soll ein neuer Space Invader ins Spiel kommen. **Hört sich kompliziert an?** Keine Sorge, das **geht in zwei Zeilen**:
 - (a) Am Anfang der Datei importiert ihr die Ship-Klasse mit `from ship import Ship`
 - (b) Wenn ein Brick getroffen wurde, hängt ihr ein neues Ship-Objekt an die Liste `Game.ships` an. Sucht dazu die Zeile mit `Wall.brick[i2, j2] = 0` (damit wird ein getroffener Brick vaporisiert) und fügt darunter
`Game.ships.append(Ship(0))`
ein.

Testet, ob das klappt: Bei jedem zerstörten Brick sollte jetzt ein neues Raumschiff ins Spiel kommen.

Wenn alles funktioniert:

```
git commit -a -m "bewegtes Zeug ist in Game"
git push
```

Schritt 7: Ganz andere Baustelle

Testet, was passiert, wenn ihr **im laufenden Betrieb den Mikrocontroller abzieht**.

Dann stürzt das Programm ab.

Laut Fehlermeldung entsteht das Problem in `controller.py`, genauer gesagt in der Methode `readline()`.

Die hat aktuell diese Struktur:

```
def readline():
    if Controller.ser is None:
        return None

    while Controller.ser.in_waiting > 0:
        # Code um die serielle Schnittstelle auszulesen

    return None
```

Was passiert hier?

Wenn der Mikrocontroller im laufenden Betrieb entfernt wird, dann ist `Controller.ser` zwar ungleich `None`, aber:

Sobald man auf das Objekt zugreift (z. B. `Controller.ser.in_waiting`), kracht es.

Die Lösung:

Wir packen die While-Schleife, in der das Objekt verwendet wird, in einen `try`-Block – und im zugehörigen `except`-Block fangen wir den Fehler ab, indem wir `Controller.ser` wieder auf `None` setzen.

Die Struktur wird also so angepasst:

```
def readline():
    if Controller.ser is None:
        return None

    try:
        while Controller.ser.in_waiting > 0:
            # Code wie zuvor

    except:
        print("disconnected")
        Controller.ser = None
    return None
```

Aufgabe

Macht diese Anpassung und testet, ob ihr jetzt den Mikrocontroller im laufenden Betrieb entfernen könnt, ohne dass das Programm abstürzt.

Hinweis: Wenn ihr `ser` übrigens nicht in eine Klasse `Controller` gepackt habt (also im Code `ser.in_waiting` statt `Controller.ser.in_waiting` vorkommt), dann sollte im `except`-Block natürlich auch einfach `ser = None` stehen.

Denkt daran: Wenn ihr den Mikrocontroller neu anschließt, müsst ihr an den Laborrechnern `usb_check` ausführen, damit er wieder erkannt wird.

Schritt 8: Zum Abschluss ein paar Bomben

Die armen Space Invader können sich gar nicht wehren. In diesem Schritt erlauben wir ihnen, **Bomben abwerfen** zu können. So richtig helfen wird ihnen das heute aber noch nicht – die Bomben werden unser Paddle (noch) nicht zerstören.

Es geht hier vor allem darum zu testen, **wie einfach sich ein neues Feature in den umorganisierten Code einbauen lässt**. Und darum, euch zu zeigen, **wie man sowas quick and dirty schrittweise implementieren kann**.

Aufgabe

1. Importiert in `ship.py` ganz oben:

```
import random
from rocket import Rocket
```

Das Modul `random` brauchen wir für Zufallszahlen, und mit `Rocket` holen wir uns unsere Klasse für Raketen.

Fügt in der `move()`-Methode der Ship-Klasse diese zwei Zeilen hinzu:

```
if random.randint(1, 100) == 1:
    Game.rockets.append(Rocket(self.x, self.y, 10))
```

Was passiert hier? Pro Frame hat jedes Raumschiff eine 1%-Chance, eine „Bombe“ abzuwerfen. Okay, streng genommen ist es keine Bombe, sondern eine Rakete – aber das sind Details, die man später noch ändern kann. 🤪

Testet, was passiert! Ihr solltet beobachten: Die Raumschiffe werfen tatsächlich Raketen nach unten ab (die erstmal noch in die falsche Richtung gehen). Aber: Beim „Angriff“ gehen sie aktuell selbst dabei drauf.

2. **Warum zerstören sich die Raumschiffe selbst?** Das passiert, weil aktuell für jedes Objekt in `Game.rockets` geprüft wird, ob der Abstand zu einem Raumschiff klein genug für einen Kontakt ist. **Die Lösung:** Wir legen in `Game` eine neue Liste `Game.bombs` an, damit Bomben getrennt von Raketen verwaltet werden.

(a) Fügt in `Game` eine neue, leere Liste `bombs` hinzu. Also analog zu den bisherigen Listen `balls`, `rockets` und `ships`.

(b) Ändert die `draw`-Methode von `Game`, sodass

- alle Objekte in `Game.balls`, `Game.rockets`, `Game.ships` und jetzt auch in `Game.bombs` bewegt und gezeichnet werden.
- Objekte, die nicht mehr am Leben sind, aus `Game.bombs` entfernt werden.

(c) Ändert den Bomben-Abwurf in `ship.py`, sodass die Raketen bei `Game.bombs` angehängt werden.

Testet danach das Spiel! Die Space Invader sollten jetzt ihren Bombenabwurf überleben.

3. Es ist natürlich noch unschön, dass aktuell einfach Raketen statt Bomben abgeworfen werden. Kein Problem – ihr wisst inzwischen, wie man sowas analog löst!

Erzeugt euch mit `cp rocket.py bombs.py` eine Kopie von `rocket.py`.

In `bombs.py` macht ihr dann folgende Änderungen:

(a) Die Klasse nennt ihr `Bomb`.

(b) Als Bild verwendet ihr `data/bomb.png`.

(c) In der `move`-Methode stellt ihr sicher, dass `self.alive` auf `False` gesetzt wird, wenn `self.y > Game.height` gilt.

Und in `ship.py` importiert ihr jetzt die Klasse `Bomb` statt `Rocket`, und ihr hängt auch ein `Bomb`-Objekt an, statt eines `Rocket`-Objekts.

Testet, ob jetzt tatsächlich Bomben fliegen! Wenn alles klappt, darf euer Code ins Repository:

```
git add bombs.py
git commit -a -m "Bomben Stimmung"
git push
```